

University Research System

1. Write a query to display all researchers along with the projects they work on. Include researchers who are not currently working on any project.
2. List all research projects with their lead researcher's full name and email. Show only projects that have at least one grant funding them.
3. Write a query to show all possible combinations of researchers and publications. Then explain why this might be a bad idea for a production query.
4. Display all researchers who supervise others, along with the names of researchers they supervise. Include the supervision start date and role.
5. Write a query to find all researchers who have published papers but are NOT currently working on any active project.
6. Retrieve the five most-cited publications, ensuring that any publications sharing the same citation count as the fifth-ranked entry are also included.
7. Retrieve researchers ordered by last name, displaying the second page of results with 10 records per page.
8. Compare the results of:

```
SELECT TOP 3 * FROM Researcher ORDER BY LastName
```

Versus:

```
SELECT * FROM Researcher ORDER BY LastName OFFSET 0 ROWS FETCH NEXT 3 ROWS ONLY
```

Are they functionally identical? What are the advantages of OFFSET-FETCH over TOP?

9. Assign a unique sequential index to each researcher within the same building, ordered by date of birth from oldest to youngest.
10. Rank publications within each publication type (Journal or Conference) based on their citation count, and display the publication title, type, citation count, and rank.
11. Rank research projects by their budget within each status category, then explain how ranking with gaps differs from ranking without gaps.
12. Divide researchers into four equal groups based on the number of publications they have authored, and display each researcher's ID, name, publication count, and group number.
13. Explain the logical execution order of the following query clauses: SELECT, FROM, WHERE, GROUP BY, HAVING, ORDER BY, TOP/OFFSET-FETCH.

14. Given this query, explain why it produces an error and how to fix it:

```
SELECT ResearcherId, COUNT(*) as ProjectCount
FROM WorksOn
WHERE ProjectCount > 2
GROUP BY ResearcherId
```

15. Write a query to display each researcher's full name (FirstName + MiddleName + LastName) as a single column, their email in lowercase, and their age in years.
16. For each research project, calculate the duration in days between StartDate and EndDate (or current date if EndDate is NULL). Also show the month and year when the project started.
17. Write a query to find all researchers whose email domain (part after @) is 'university.edu'.
18. Display researcher details, substituting 'N/A' wherever the MiddleName or RoomNumber is missing.
19. Write a query to classify projects into categories based on their budget: 'Small' for budgets under 50,000, 'Medium' for budgets between 50,000 and 150,000, and 'Large' for budgets exceeding 150,000.
20. Calculate the total budget managed by each researcher who leads one or more projects, and display their full name alongside this total.
21. Display the number of researchers in each building, but only for buildings that have more than 3 researchers.
22. For each publication type, calculate the average citation count, total citations, and number of publications. Filter to show only types with an average citation count above 50.
23. Find researchers who work on more than 2 projects and have a total weekly hour commitment exceeding 60 hours. Show ResearcherId, project count, and total hours.
24. Write a query using a subquery to find all projects that have a budget greater than the average budget of all projects.
25. Write a query using a subquery to find researchers who have more publications than the average number of publications for researchers in the same building.
26. Write a query using EXISTS to find all researchers who supervise at least one other researcher.
27. Use a subquery to display each project along with the total number of researchers working on it.
28. Write a query to find the second highest budget among all research projects (Do not use OFFSET-FETCH or ranking functions).
29. Combine the list of all researcher emails and all grant funding agency names into a single list labeled "Contact Information".

30. Identify researchers involved in projects who have not authored any publications.
31. Find researchers who supervise others and also lead one or more research projects.
32. Write a CTE to calculate the total hours per week for each researcher across all their projects, then use it to find researchers working more than 40 hours per week.
33. Write a query using multiple CTEs: one to get the total budget per researcher (who leads projects), another to get their publication count, then join them to show researchers with budget > 100000 AND at least 3 publications.
34. Write a reusable function that takes a ProjectId and returns how many days the project has been active, calculating from the start date to either the end date or today if the project is ongoing.
35. Write a reusable function that takes a ResearcherId and returns all projects they are involved in, showing the project title, their role, and hours worked per week.
36. Write a function that takes a ResearcherId and returns a table containing the total number of projects, total publications, total hours worked per week, and average citations per publication for that researcher. Also, explain the scenarios where a multi-statement table-valued function is preferred over an inline table-valued function.
37. Create a non-clustered index on the ResearchProject table to improve queries that search by Status and StartDate.
38. Explain the difference between a clustered and non-clustered index. How many clustered indexes can a table have? What happens to existing indexes when you create a clustered index?
39. You have a query that frequently searches for researchers by Email and retrieves their FirstName and LastName. Write a covering index that would make this query more efficient. Explain what a covering index is and why it improves performance.
40. Create a view named ActiveProjectSummary that shows project title, leader name, number of team members, total hours per week allocated, and total budget.
41. Create an indexed view named ResearcherPublicationStats that shows ResearcherId, researcher full name, and total number of publications. Include the necessary.
42. Explain the requirements and restrictions for creating an indexed view. What are the performance benefits? When would you choose an indexed view over a regular view or a table?
43. Create a stored procedure named AddResearcherToProject that accepts ResearcherId, ProjectId, JoinDate, Role, and HoursPerWeek as parameters. The procedure should:
 - Validate that both researcher and project exist.
 - Check that the researcher isn't already on the project.
 - Insert the record into WorksOn.
 - Return 0 for success, -1 for errors.

44. Create a stored procedure named UpdateProjectStatus that accepts a ProjectId and changes its status from 'Pending' to 'Active', but only if the project has at least one researcher assigned and at least one funding source. Use appropriate error handling.
45. Write a stored procedure with OUTPUT parameters that accepts a ResearcherId and returns the total number of projects they work on, their total publications, and their total weekly hours across all projects.
46. Create an AFTER INSERT trigger on the WorksOn table that prevents a researcher from being assigned to more than 5 projects. If the insertion would exceed this limit, rollback the transaction and raise an error.
47. Create a trigger on the ResearchProject table that automatically updates the Status to 'Completed' when an EndDate is set to a date in the past. Should this be an AFTER or INSTEAD OF trigger? Explain your choice.
48. Create an audit trigger that logs all updates to the Grants table. Create an appropriate audit table to store: GrantId, OldAmount, NewAmount, ModifiedBy (SYSTEM_USER), ModifiedDate.
49. Write a transaction that:
 - Creates a new research project.
 - Assigns the project leader to work on it.
 - Allocates a grant to fund it.

Include proper error handling with TRY-CATCH and ROLLBACK if any step fails.

50. Explain the ACID properties of transactions. For each property, give an example from the university research database showing why it's important.
51. Write the necessary GRANT statements to:
 - Give user 'ResearchManager' full permissions on all tables.
 - Give user 'ResearchAssistant' SELECT and INSERT permissions only on Researcher and Publication tables.
 - Give user 'DataAnalyst' SELECT permission on all views but no direct table access.
52. Write REVOKE statements to remove INSERT and UPDATE permissions from user 'ResearchAssistant' on the Researcher table. Explain the difference between GRANT, REVOKE, and DENY.

53. Compare these two queries for finding researchers who work on project 'P001':

Query A:

```
SELECT r.*
FROM Researcher r
WHERE r.Id IN (
    SELECT ResearcherId
    FROM WorksOn
    WHERE ProjectId = 'P001'
)
```

Query B:

```
SELECT r.*
FROM Researcher r
WHERE EXISTS (
    SELECT 1
    FROM WorksOn w
    WHERE w.ResearcherId = r.Id
    AND w.ProjectId = 'P001'
)
```

Which query is likely to perform better and why? What factors influence the optimizer's choice?

54. A query retrieving all projects with their researchers is running slowly:

```
SELECT p.Title, r.FirstName, r.LastName
FROM ResearchProject p
LEFT JOIN WorksOn w ON p.Id = w.ProjectId
LEFT JOIN Researcher r ON w.ResearcherId = r.Id
WHERE p.Status = 'Active'
```

```
ORDER BY p.StartDate DESC
```

Suggest at least 3 different optimizations (indexes, query rewrite, etc.) that could improve performance.

55. You need to regularly retrieve the top 10 most cited publications along with their authors. You have four options:

- Option A: Write the query each time it's needed.
- Option B: Create a view.
- Option C: Create a stored procedure.
- Option D: Create an indexed view.

Discuss the pros and cons of each approach. Which would you choose and why? Consider factors like performance, maintenance, and data freshness.